

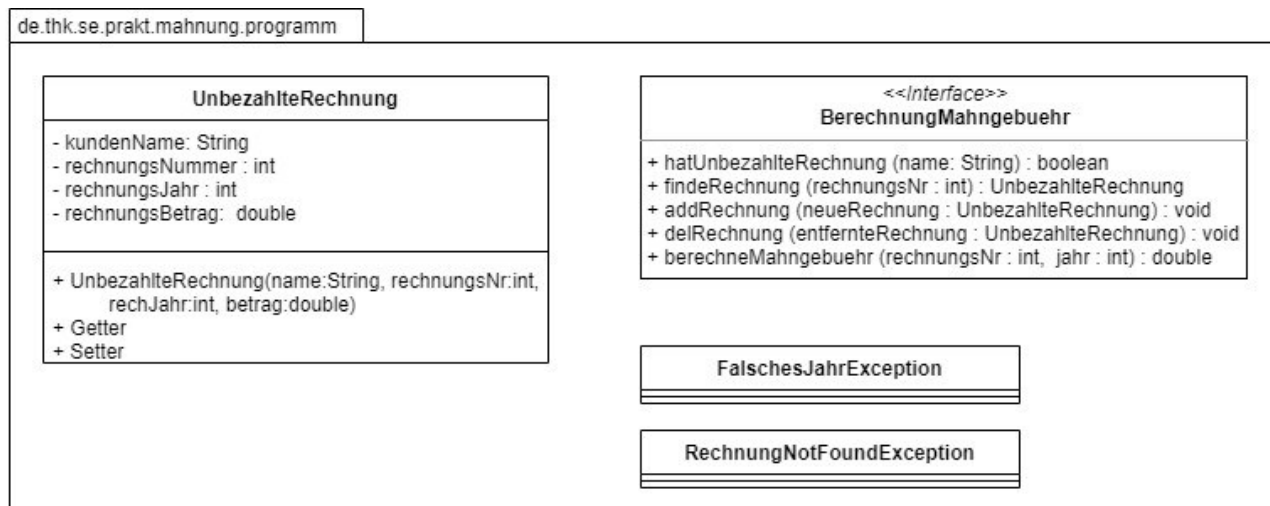
Software Engineering

Praktikumsversuch 2, Gruppe C - Hausaufgaben -

Ziel: Erstellung von JUnit-Testfällen

Abgabe der Lösungen: Bis zum 13.12., 08:00 Uhr morgens, im Master-Branch Ihres Gitlab-Repositories P2SE<IhreTeamnummer>. Abzugeben ist das vollständige IntelliJ-Projekt. In Ihrem Gitlab-Repository zu diesem Praktikumsversuch habe ich Ihnen ein IntelliJ-Projekt zur Verfügung gestellt, welches Sie als Grundlage Ihrer Implementierung verwenden müssen.

Es soll ein kleines Programm zur Berechnung der Mahngebühren für unbezahlte Rechnungen erstellt werden. Es existiert hierfür das folgende Klassendiagramm:



Eine unbezahlte Rechnung besitzt als Attribute den Namen des Kunden, die eindeutige Rechnungsnummer, das Rechnungsjahr und den Rechnungsbetrag.

Bereits implementiert sind die Exception-Klassen, die Klasse `UnbezahlteRechnung` und das Interface `BerechnungMahngebuehr`.

Ihre Aufgaben:

H 2.1 Implementierung der Schnittstellen-Klasse

Erstellen Sie eine Implementierung der Interface-Klasse `BerechnungMahngebuehr`.

Diese Implementierung soll eine private `ArrayList` mit den bisher unbezahlten Rechnungen besitzen (im Folgenden als Rechnungsliste bezeichnet).

Die Semantik der Methoden des Interface ist:

- Die Methode `hatUnbezahlteRechnung` nimmt den Namen eines Kunden als Parameter und liefert `true`, falls eine Rechnung mit diesem Kundennamen in der Rechnungsliste existiert und `false`, falls keine Rechnung mit diesem Kundennamen in der Rechnungsliste existiert.
- Die Methode `findeRechnung` nimmt eine Rechnungsnummer als Parameter und liefert die unbezahlte Rechnung aus der Rechnungsliste mit dieser Rechnungsnummer als Ergebnis. Sie wirft die Exception `RechnungNotFoundException`, falls sich keine Rechnung mit dieser Rechnungsnummer in der Rechnungsliste befindet.
- Die Methode `addRechnung` fügt eine unbezahlte Rechnung zur Rechnungsliste hinzu.
- Die Methode `delRechnung` entfernt eine unbezahlte Rechnung aus der Rechnungsliste.
- Die Methode `berechneMahngebuehr` nimmt eine Rechnungsnummer und ein Jahr als Parameter und berechnet die Mahngebühr für diese Rechnung für das übergebene Jahr:
 - Bei einer Rechnung mit einem Rechnungsjahr welches exakt dem übergebenen Jahr entspricht, beträgt die Mahngebühr 5% des Rechnungsbetrags,
 - Falls das Rechnungsjahr der Rechnung exakt 1 Jahr vor dem übergebenen Jahr liegt, beträgt die Mahngebühr 8% des Rechnungsbetrags,
 - Falls das Rechnungsjahr der Rechnung exakt 2 Jahre vor dem übergebenen Jahr liegt, beträgt die Mahngebühr 10% des Rechnungsbetrags,
 - Falls das Rechnungsjahr der Rechnung 3 oder 4 Jahre vor dem übergebenen Jahr liegt, beträgt die Mahngebühr 14% des Rechnungsbetrags,
 - Falls das Rechnungsjahr der Rechnung 5 oder mehr Jahre vor dem übergebenen Jahr liegt, beträgt die Mahngebühr 100% des Rechnungsbetrags.
- Die Methode `berechneMahngebuehr` wirft die Exception `FalschesJahrException`, falls das übergebene Jahr echt kleiner als das Rechnungsjahr ist.
- Die Methode `berechneMahngebuehr` reicht die geworfene Exception `RechnungNotFoundException` an den Aufrufer weiter.

H 2.2 Realisierung von JUnit-Testfällen

Verwenden Sie bitte nur die im zur Verfügung gestellten Projekt enthaltenen Bibliotheken und keine anderen.

Erstellen Sie im Paket `de.thk.se.prakt.mahnung.test` JUnit-Testklassen zur Prüfung der folgenden Eigenschaften Ihrer Implementierung.

Schreiben Sie bitte vor jeden JUnit-Testfall in Ihrem Code einen Java-Kommentar mit der Nummer der jeweiligen Aufgabe, also bspw. `// (3)`.

Hinweis: Die Methoden `addRechnung()` und `delRechnung()` aus der zu testenden Klasse dürfen in den Testfällen zur Herstellung des gewünschten Zustands verwendet werden.

- (1) Wenn eine unbezahlte Rechnung mit dem Kundennamen „Jeff“ in der Rechnungsliste vorhanden ist, liefert der Methodenaufruf `hatUnbezahlteRechnung („Jeff“)` `true`.

- (2) Wenn eine unbezahlte Rechnung mit dem Kundennamen „Jeff“ in der Rechnungsliste nicht vorhanden ist, liefert der Methodenaufruf `hatUnbezahlteRechnung („Jeff“)` `false`.
- (3) Wenn keine Rechnung mit der Nummer 68 in der nicht-leeren Rechnungsliste existiert, wirft der Methodenaufruf `findeRechnung(68)` die Exception `RechnungNotFoundException`.
- (4) Wenn die Rechnung mit der Nummer 68 in der nicht-leeren Rechnungsliste existiert, liefert der Methodenaufruf `findeRechnung(68)` genau diese Rechnung.
- (5) Wenn die Rechnungsliste leer ist, wirft der Methodenaufruf `findeRechnung(68)` die Exception `RechnungNotFoundException`.
- (6) Für eine Rechnung mit der Rechnungsnummer 66 und dem Rechnungsjahr 2021 wirft der Methodenaufruf `berechneMahngebuehr(66, 2020)` die Exception `FalschesJahrException`.
- (7) Implementieren Sie parametrisierte Testfälle für die folgenden Prüfungen:
 - Für eine in der Rechnungsliste existierende unbezahlte Rechnung mit Nummer 89, mit dem Rechnungsjahr 2021 und dem Rechnungsbetrag 478,45 liefert der Aufruf von `berechneMahngebuehr (89, 2021)` das Ergebnis 23.9225.
 - Für eine in der Rechnungsliste existierende unbezahlte Rechnung mit Nummer 57, mit dem Rechnungsjahr 2020 und dem Rechnungsbetrag 4783,33 liefert der Aufruf von `berechneMahngebuehr (57, 2021)` das Ergebnis 382.6664.
 - Für eine in der Rechnungsliste existierende unbezahlte Rechnung mit Nummer 96, mit dem Rechnungsjahr 2018 und dem Rechnungsbetrag 2999,99 liefert der Aufruf von `berechneMahngebuehr (96, 2020)` das Ergebnis 299.999.
 - Für eine in der Rechnungsliste existierende unbezahlte Rechnung mit Nummer 1, mit dem Rechnungsjahr 2010 und dem Rechnungsbetrag 7296,67 liefert der Aufruf von `berechneMahngebuehr (1, 2014)` das Ergebnis 1021.5338.
 - Für eine in der Rechnungsliste existierende unbezahlte Rechnung mit Nummer 49, mit dem Rechnungsjahr 1999 und dem Rechnungsbetrag 12798,60 liefert der Aufruf von `berechneMahngebuehr (49, 2021)` das Ergebnis 12798.60.
- (8) Erstellen Sie im Paket `de.thk.se.prakt.mahnung.test` eine JUnit-Test-Suite, durch die alle erstellten Testklassen aufgerufen werden.